ChatGPT

Proyecto de PMS multisitio basado en HAIP para Justin, Jurassic y operación de motel en Barranquilla

Resumen ejecutivo

Después de revisar el repositorio oficial de **HAIP — Hotel AI Platform**, su arquitectura, licencia, módulos y estado actual, mi conclusión es que **sí es una base técnicamente razonable para construir el sistema que necesita tu hermana**, y resulta mucho más conveniente partir de HAIP que desarrollar desde cero todo un PMS hotelero. HAIP ya resuelve buena parte del dominio difícil: reservas, huéspedes, habitaciones, folios/cargos, caja, housekeeping, usuarios y roles, auditoría, tarifas, fidelización, WebSockets en tiempo real, importación CSV, exportación contable, webhooks e incluso puntos de extensión para cerraduras. [1]

Además, la licencia **Apache-2.0 permite usar, modificar y distribuir comercialmente el software y crear obras derivadas**. Al redistribuirlo deben conservarse la licencia y avisos aplicables, y los archivos modificados deben indicar que fueron cambiados; la licencia tampoco concede automáticamente derechos sobre las marcas del autor. Por tanto, pueden crear su propia versión, nombre, interfaz y módulos sin quedar obligados a publicar sus modificaciones, siempre respetando esas condiciones y revisando también las licencias de dependencias de terceros. [2]

Mi recomendación no sería convertir HAIP en un gigantesco monolito específico de Justin/Jurassic. Mantendría su **núcleo PMS relativamente cercano al upstream** y construiría encima una capa propia para Colombia y para la operación motelera:

> **HAIP Core + módulos propios + Front Desk Touch + PWA móvil + Edge Gateway por establecimiento + adaptadores de cerraduras/cámaras/contabilidad.**

El punto arquitectónico más importante es este: **no recomiendo tres bases PostgreSQL completas replicándose bidireccionalmente entre sí**. HAIP ya está diseñado como multi-property, con `property_id` y aislamiento por establecimiento; para solo dos ubicaciones físicas es más seguro mantener una **base central canónica** y tener en cada hotel una pequeña capa local de contingencia/offline que almacene caché operativo y una cola de operaciones pendientes. HAIP ya tiene un modelo multi-tenant, WebSockets por propiedad, webhooks de eventos e integración por adaptadores, por lo que esta arquitectura encaja bastante bien con el producto existente. [3]

La organización empresarial que propongo es separar **ubicación física** de **unidad operativa**:

Ubicación física	Unidad lógica PMS	Modalidad
Justin	Justin Hotel	Alojamiento por noche

Ubicación física	Unidad lógica PMS	Modalidad
Justin, si allí está la zona motel	Justin Motel	Estancias cortas/por horas
Jurassic	Jurassic Hotel	Alojamiento por noche

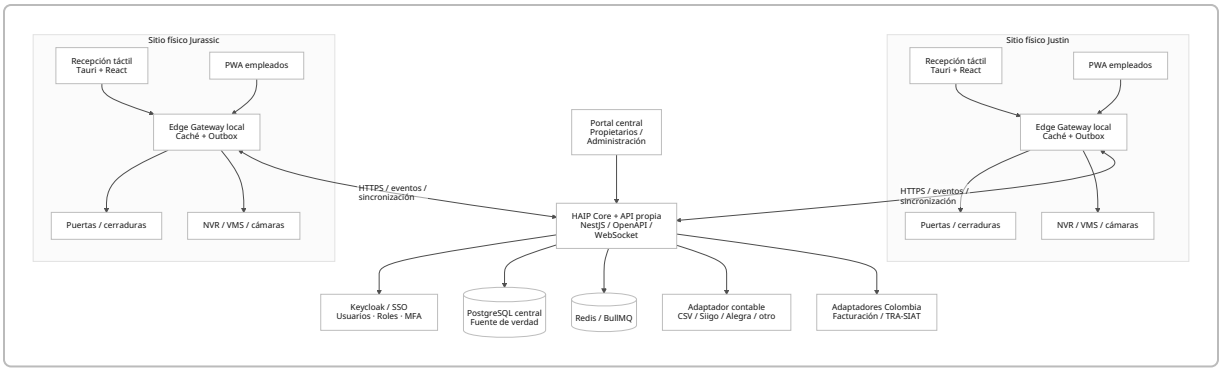
Si la zona motel está realmente dentro de Jurassic, simplemente se invierte la relación. El beneficio de hacer **tres unidades lógicas aunque solo existan dos edificios** es poder separar tarifas, cierres, indicadores, inventarios, políticas, permisos, comisiones, reportes y cumplimiento. Esto cobra especial importancia en Colombia porque MinCIT distingue expresamente el alojamiento turístico de los establecimientos de hospedaje por horas: los hoteles forman parte de los prestadores turísticos, mientras que MinCIT indica que los establecimientos de hospedaje por horas no requieren RNT al no considerarse prestadores de servicios turísticos. 4

Eso no significa que necesariamente deban existir tres empresas, tres NIT o tres resoluciones de facturación. Esa parte depende de cómo esté estructurado hoy el negocio y debe validarse con el contador y asesor jurídico antes de parametrizar la facturación.

La visión final sería esta:

- Tu hermana entra a un **Portal Central** y ve Justin + Jurassic + la unidad motelera, ventas, caja, habitaciones, ocupación, inventario, comisiones, auditoría y alertas.
- Cada recepción usa una **aplicación táctil extremadamente sencilla**, sin menús innecesarios.
- Cada empleado ve únicamente lo autorizado por su rol.
- Personal de aseo/revisión utiliza una **PWA en el celular** para limpieza, consumos e inventario.
- Cerraduras y cámaras se integran a través de un **Gateway local**, sin entregar credenciales de infraestructura física al navegador.
- Si Internet cae, recepción continúa con funciones críticas y sincroniza posteriormente.
- Contabilidad recibe asientos/documentos mediante CSV inicialmente y posteriormente mediante API.
- Todos los eventos delicados —descuentos, apertura manual de puertas, anulaciones, diferencias de caja, modificación de consumos, consultas de cámaras— quedan auditados.

Una arquitectura de este tipo es sustancialmente más importante que simplemente “hacer un software de hotel”. En este negocio el software se convierte también en **sistema de control interno**.



La inversión de desarrollo no es pequeña. Para una solución verdaderamente operativa —no solamente una demo— estimo aproximadamente **1.300–2.400 horas de ingeniería** para el alcance completo descrito, incluyendo offline, hardware, cámaras, migración, seguridad y pruebas. Un MVP online-first puede salir con bastante menos; la integración de hardware desconocido y la operación offline completa son los dos factores que más pueden ampliar el esfuerzo.

Evaluación de HAIP como punto de partida

Al **25 de octubre de 2026**, el repositorio oficial identifica HAIP como un PMS open-source y API-first construido con TypeScript, Node.js y NestJS. La última release publicada que aparece en GitHub es **v3.0.7, del 19 de octubre de 2026**, después de una secuencia frecuente de releases 3.0.x durante octubre, lo cual indica mantenimiento reciente y activo. Precisamente por esa velocidad de cambios conviene congelar un tag exacto para el proyecto de Justin/Jurassic y no desplegar directamente `main`. [5]

El proyecto se presenta como un PMS completo y enumera reservas, facturación/folios, tarifas, housekeeping, auditoría nocturna, grupos, caja, contabilidad, canales, motor de reservas, proveedores de pago, autenticación Keycloak, usuarios/roles, impuestos, facturación fiscal, registro de huéspedes, integraciones y hooks para cerraduras. [6]

Stack observado en el código:

Capa	HAIP actual	Utilidad para el proyecto
Backend	NestJS 10.x / TypeScript	Excelente para módulos propios
Base de datos	PostgreSQL + Drizzle ORM	Fuente central del PMS
Cache/colas	Redis + BullMQ	Procesos, sincronización y jobs
Frontend	React + Vite	Compatible con recepción/PWA
Tiempo real	Socket.IO	Estados de habitaciones instantáneos
Autenticación	Keycloak + JWT + roles locales	SSO y RBAC
API	REST + OpenAPI	Cerraduras, móvil, contabilidad
Integraciones	Registry + webhooks	Muy útil para adapters
Infraestructura	Docker / Compose	Desarrollo y despliegue
Testing	Vitest	Extender pruebas del negocio

El `package.json` actual del API muestra NestJS 10.4, BullMQ 5.81, Drizzle ORM 0.39, Socket.IO y PostgreSQL. [7]

Hay un detalle que yo pondría como **primer ticket técnico del proyecto**: la documentación y los manifiestos no están totalmente alineados. El README declara React 19 y Tailwind 4, mientras que `apps/dashboard/package.json` actualmente fija React 18.3.1, React Router 6.28, Tailwind 3.4.15 y Vite 6. [8]

Esto no impide usar HAIP, pero significa que Claude o los desarrolladores deben basarse en el **lockfile y tag seleccionado**, no asumir versiones a partir del README.

HAIP declara **2.553 pruebas en 294 archivos con pruebas exitosas** en su README actual. Es una señal positiva, aunque no sustituye las pruebas específicas de Justin/Jurassic ni una auditoría propia antes de producción. [9]

Qué ya trae y qué aprovecharía

Necesidad	HAIP actual	Acción
Multi-hotel	Sí, `property_id`	Reutilizar
Reservas por noche	Sí	Reutilizar
Walk-in/check-in/check-out	Sí	Reutilizar
Estados habitaciones	Sí	Personalizar UX
Housekeeping	Sí	Reutilizar/ampliar
Cargos/folios	Sí	Reutilizar
Caja/cash drawer	Sí	Reutilizar
Roles/permisos	Sí	Extender granularidad
Tiempo real	Sí, Socket.IO	Reutilizar
Auditoría	Base existente	Ampliar
Fidelización	Sí, loyalty ledger	Localizar/reglas propias
Minibar como cargo	Sí, inbound folio posting	Extender a inventario físico
Importación	CSV existente	Extender
Exportación contable	CSV existente	Adaptar Colombia
Cerraduras	Hooks/adapters	Crear proveedor real
Cámaras	No como PMS central	Crear módulo/gateway
Hospedaje por minutos/horas	No como flujo nativo observado	Desarrollar
Comisión taxistas/conductores	No	Desarrollar
Offline por sitio	No como PMS edge completo	Desarrollar
PWA operacional	No específica	Desarrollar
App táctil kiosk	No específica	Desarrollar

HAIP ofrece un importador CSV con mapeo de columnas, dry-run y errores por fila, y exportación CSV del diario de ingresos y balance de comprobación. También tiene un mecanismo idempotente de cargos entrantes al folio que el propio proyecto pone como ejemplo para teléfono/minibar. [10]

También existe ya un ledger de fidelización con cuentas y movimientos earn/burn/adjust, por lo que **no desarrollaría un loyalty engine desde cero**. Lo extendería con campañas, reglas y consentimiento de marketing apropiados para Colombia. [11]

La arquitectura de autenticación también es una muy buena base: HAIP usa Keycloak/OIDC/JWT y superpone usuarios, roles y permisos propios por propiedad. El README describe permisos que controlan tanto endpoints como navegación de la interfaz. [12]

Un aspecto que sí debe corregirse desde el comienzo es el despliegue: el `docker-compose.yml` de demostración tiene autenticación desactivada y pagos simulados, mientras que la documentación separa un overlay de producción con autenticación habilitada. **Nunca desplegaría la configuración demo en los hoteles.** [13]

Finalmente, HAIP ya transmite eventos como `room.status_changed`, reservas y housekeeping mediante Socket.IO, y su modelo de webhooks permite que sistemas externos reaccionen a cambios del PMS. Ese mecanismo es ideal para sincronizar Front Desk, PWA, cerraduras, contabilidad y otras integraciones sin acoplar todo el código. [14]

Operación propuesta y experiencia de usuario

El software debe modelar no solamente “reservas”, sino el ciclo real de dinero y responsabilidad dentro del establecimiento.

La pregunta principal de cada evento operativo debe ser:

> **¿quién hizo qué, en qué habitación, cuándo, con qué autorización y cuál fue el efecto económico?**

Ese principio permite controlar descuentos, inventario, caja, empleados, aperturas de puerta y comisiones.

Recepción táctil

La pantalla principal no debe parecerse al panel administrativo convencional de un PMS. Para recepción diseñaría un **Room Board táctil** con botones grandes y estados inmediatamente reconocibles.

Cada habitación mostraría como mínimo:

Información visible	Ejemplo
Número	205
Estado	Libre / Ocupada / Aseo

Información visible	Ejemplo
Modalidad	Hotel / Horas
Tiempo	`01:42 transcurrido`
Próximo evento	`Salida 14:00`
Pendiente minibar	`3 productos`
Incidencia	`Mantenimiento`
Acceso	`Puerta OK`

Los estados operacionales deberían normalizarse, por ejemplo:

`LIBRE_LIMPIA → OCUPADA → SALIDA_PENDIENTE → SUCIA → EN_ASEO → INSPECCION → LISTA`

y estados laterales:

`RESERVADA`, `BLOQUEADA`, `MANTENIMIENTO`, `FUERA_DE_SERVICIO`.

HAIP ya cuenta con máquina de estados de habitaciones y housekeeping, por lo que la intención sería mapear esta experiencia simplificada sobre su dominio existente, no crear una segunda fuente de verdad. [15]

Flujo de motel / estancia corta:



La tarifa **no debería modelarse simplemente como una reserva con fechas alteradas**. Crearía una entidad específica `short_stay_session`, porque una operación por bloques de minutos necesita reglas diferentes:

```
short_stay_session
- id
- property_id
- physical_site_id
- room_id
- guest_id opcional/según requisito
- started_at
- scheduled_end_at
- actual_end_at
- billing_policy_id
- billing_unit_minutes
- grace_period_minutes
```

```
- extension_count
- referral_id
- access_credential_id
- folio_id
- status
```

El motor tarifario debería soportar **15/30/60 minutos como unidades configurables**, pero no codificarlas rígidamente. Es mejor permitir reglas como:

- paquete inicial de X minutos;
- valor por bloque adicional;
- período de gracia;
- redondeo;
- extensión manual;
- tarifa por franja horaria;
- tarifa fin de semana;
- habitación/tipo de habitación;
- promoción;
- autorización requerida para precio manual.

Así la administración puede cambiar precios sin pedir una modificación del programa.

Los descuentos deben tener cuatro niveles: **automático por tarifa/promoción, permitido al recepcionista dentro de un límite, autorizado por supervisor y excepcional con doble control**. Ningún descuento manual debería desaparecer dentro del total: debe quedar `precio_original`, `descuento`, `motivo`, `usuario`, `autorizador`, `timestamp`.

Origen del huésped y comisiones a conductores

No utilizaría simplemente un campo de texto “taxista”. Crearía un pequeño módulo de referidos:

```
referrer
- id
- type: taxi | plataforma | conductor_particular | convenio
- alias/nombre
- teléfono opcional
- placa opcional
- status
- commission_scheme_id

referral
- stay_id / short_stay_session_id
- referrer_id
- captured_by_user_id
- amount_or_rule
- evidence/reference
```

```
- approved_by
- settlement_status
```

La comisión debe registrarse separadamente de la venta al huésped. Así se puede responder:

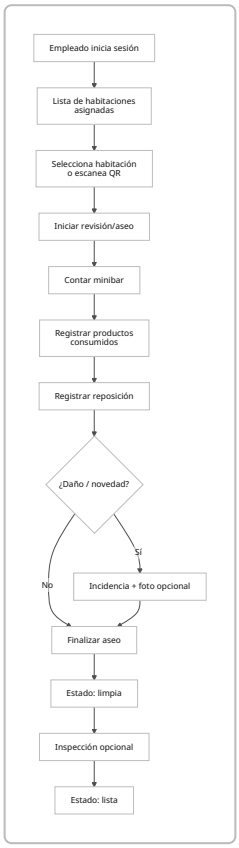
- cuánto produjo cada canal;
- cuánto se pagó en comisiones;
- quién cargó la comisión;
- quién la aprobó;
- qué conductor tiene saldo;
- qué recepcionista genera más referidos;
- si hay comisiones duplicadas;
- rentabilidad neta por origen.

Un caso interesante es que el conductor sea desconocido. La recepción podría registrarlo rápidamente mediante placa/alias y posteriormente un supervisor unificar duplicados.

PWA móvil para aseo, minibar e inventario

La PWA debe mostrar únicamente las habitaciones asignadas al trabajador y la información mínima necesaria. No es apropiado que una persona de housekeeping vea el historial completo del huésped.

Flujo:



El inventario debe manejar **ubicaciones**, no solamente cantidades globales:

`Almacén Justin → Habitación 205 → Producto X`

y movimientos:

`compra`, `traslado`, `reposición`, `consumo`, `devolución`, `merma`, `ajuste`.

Un consumo detectado debería producir simultáneamente:

1. movimiento de stock;
2. `room_consumption`;
3. cargo al folio;
4. reposición pendiente;
5. evento de auditoría.

El mecanismo de folio entrante idempotente que ya existe en HAIP es especialmente útil para evitar cobrar dos veces un minibar cuando una PWA reintenta una operación después de perder conectividad. [16]

Fidelización y promociones

Aprovecharía el loyalty ledger existente de HAIP, pero agregaría:

• puntos por gasto;
• noches/visitas;
• beneficios configurables;
• promociones por frecuencia;
• cupones;
• promociones de cumpleaños cuando proceda;
• campañas;
• cartera de beneficios;
• niveles;
• expiración;
• reglas antifraude;
• consentimiento separado de marketing.

Dada la naturaleza particularmente privada de una estancia en motel, **no utilizaría historial detallado de visitas, cámaras o accesos para segmentación comercial**. La fidelización debería poder operar con identificador/cliente y saldo de puntos, mientras que el acceso al detalle histórico queda restringido.

Control interno que añadiría desde el primer día

Hay varios controles que probablemente les generen más retorno que funciones llamativas:

Control	Qué evita/detecta
Apertura y cierre de turno	Caja sin responsable

Control	Qué evita/detecta
Fondo inicial/final	Diferencias de efectivo
Arqueo ciego	Manipulación del “esperado”
Descuento con autorización	Descuentos fraudulentos
Anulación con motivo	Borrar consumos
Historial de precios	Cambio indebido de tarifas
Log de puertas	Aperturas no asociadas
Log de consulta cámaras	Uso indebido de videovigilancia
Inventario teórico vs físico	Pérdidas/minibar
Reposición por empleado	Trazabilidad
Habitación ocupada vs PMS	Ocupación no registrada
Comisión vs estancia	Comisiones falsas
Placa/referido duplicado	Pago doble
Cierre diario por unidad	Comparación Justin/Jurassic
Bitácora inmutable de eventos	Responsabilidad
Alertas de excepción	Supervisión remota

HAIP ya dispone de caja, roles, auditoría, housekeeping y eventos que reducen bastante el trabajo para implementar estos controles. [17]

Arquitectura, offline, seguridad y cumplimiento colombiano

Base central versus una base por hotel

Consideraría estas tres alternativas:

Arquitectura	Ventajas	Problemas	Veredicto
BD completa por hotel + réplica bidireccional	Cada sede muy autónoma	Conflictos, IDs, doble escritura, reconciliación contable, mantenimiento	No inicialmente
BD central únicamente	Muy simple y consistente	Internet se vuelve crítico	Insuficiente
BD central + Edge Gateway local	Consistencia + contingencia	Hay que programar sincronización	**Recomendada**

HAIP ya fue construido con `property_id` en las tablas y con aislamiento por propiedad, por lo cual una base central es coherente con su diseño. [18]

El **Edge Gateway** de Justin y Jurassic sería un pequeño servicio local que:

- mantiene el último estado útil de habitaciones;
- conserva permisos mínimos necesarios para contingencia;
- habla con las cerraduras;
- habla con NVR/VMS;
- recibe operaciones PWA/reception durante un corte;
- mantiene una `outbox`;
- sincroniza al volver Internet.

No replicaría tablas PostgreSQL directamente. Sincronizaría **comandos/eventos de dominio con IDs idempotentes**.

Por ejemplo:

```
{
  "eventId": "01J...",
  "deviceId": "JUSTIN-RECEPTION-01",
  "propertyId": "...",
  "type": "minibar.consumption_recorded",
  "occurredAt": "...",
  "payload": {},
  "idempotencyKey": "..."
}
```

El servidor registra si ya procesó ese `idempotencyKey`, evitando dobles cargos.

Qué permitir offline:

Operación	Offline
Ver último tablero de habitaciones	Sí
Cambiar limpieza/inspección	Sí
Registrar minibar	Sí
Apertura de puerta autorizada localmente	Sí, bajo políticas
Registrar estancia corta	Sí, con ID temporal/global
Extender estancia	Sí
Consultar cámaras locales	Sí

Operación	Offline
Reserva web/OTA	No garantiza actualidad
Cambios grandes de tarifa	Preferiblemente no
Crear usuarios/roles	No
Facturación electrónica DIAN	Cola/contingencia según modalidad aplicable
Reporte corporativo consolidado	Último dato sincronizado

HAIP transmite estados en tiempo real mediante Socket.IO y dispone de webhooks por cambios de entidades, por lo que su modelo event-driven ayuda a construir esta capa. [14]

Identidad y permisos

Propondría una jerarquía aproximada:

Rol	Alcance
Propietaria	Todo el grupo
Administrador general	Operaciones y configuración
Gerente Justin	Solo Justin
Gerente Jurassic	Solo Jurassic
Supervisor	Operación, descuentos y caja
Recepcionista	Recepción/cobros limitados
Caja	Cobros y cierres
Housekeeping	Tareas/inventario mínimo
Inventario	Stock/compras/reposición
Contabilidad	Finanzas/exportaciones
Seguridad	Cámaras/accesos autorizados
Auditor	Solo lectura + trazabilidad

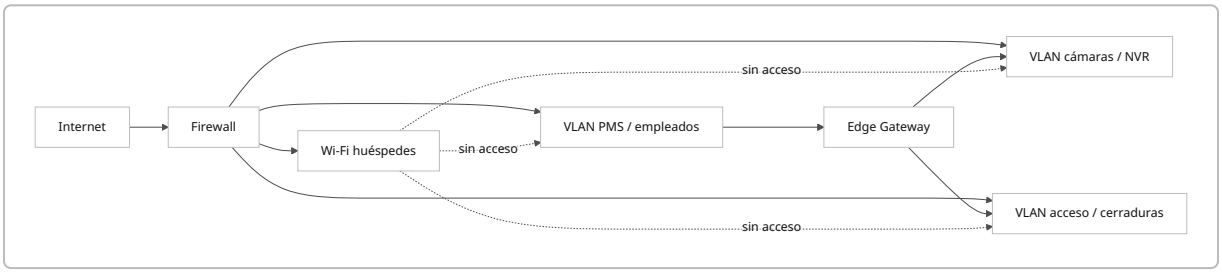
HAIP ya incorpora Keycloak OAuth/OIDC y una matriz de permisos local por propiedad, por lo que extendería esa capa en vez de crear autenticación propia. [12]

Además implementaría **step-up authentication** para operaciones de alto riesgo: abrir una puerta de forma administrativa, ver una cámara sensible, anular una venta, otorgar un descuento elevado, modificar una caja cerrada o cambiar permisos.

No utilizaría cuentas compartidas tipo `recepcion`. Una pantalla táctil puede permitir cambio de usuario rápido mediante PIN corto o credencial física, pero cada acción debe quedar vinculada a una persona concreta.

Seguridad de red

Dividiría la red de cada establecimiento aproximadamente así:



Las cámaras y controladores de puerta no deberían publicarse directamente a Internet. El Edge Gateway sería el mediador y mantendría credenciales fuera del navegador.

La instalación productiva de HAIP también debe partir de su configuración segura, no del Compose demo con `AUTH_ENABLED=false`; la documentación oficial separa explícitamente ambas configuraciones. [13]

Datos personales y videovigilancia

En Colombia, el tratamiento de datos personales está sometido al marco de la Ley 1581 de 2012 y regulación asociada. La SIC señala específicamente que los sistemas de videovigilancia implican tratamiento de datos personales y exigen especial diligencia del responsable. [19]

Por eso el PMS debería incorporar desde diseño:

- finalidad de cada categoría de datos;
- política de tratamiento;
- consentimiento cuando corresponda;
- registro de consentimiento;
- control de acceso;
- retención configurable;
- consultas/correcciones/supresión cuando jurídicamente procedan;
- log de acceso a información;
- avisos de videovigilancia;
- permisos muy restringidos sobre cámaras;
- separación entre seguridad y marketing.

Especialmente en este negocio evitaría replicar clips o fotografías de cámaras dentro de HAIP salvo que exista una necesidad concreta. El PMS debería guardar principalmente `camera_id`, zona y bitácora de acceso; el video permanece en el NVR/VMS.

Hotel versus hospedaje por horas

MinCIT indica que hoteles y otros alojamientos turísticos forman parte de los prestadores sujetos al RNT y SIAT; también indica expresamente que los **establecimientos de alojamiento por horas no requieren RNT** porque no son considerados prestadores de servicios turísticos. Asimismo, para alojamientos con PMS, MinCIT contempla el uso del SIAT/registro correspondiente dentro de la operación turística. [4]

Esa diferencia es otra razón para modelar la zona motelera como **unidad lógica separada**, aunque físicamente comparta edificio.

No automatizaría la clasificación legal solamente con un `if (duration < 24h)`: primero hay que identificar cómo están matriculados actualmente Justin y Jurassic, qué RNT poseen, qué NIT factura, cómo está registrada la actividad económica y cómo manejan la operación mixta.

Facturación electrónica

DIAN permite utilizar software propio, un proveedor tecnológico autorizado o su solución gratuita. Quien desarrolla software propio debe pasar por el proceso de habilitación/pruebas correspondiente. [20]

Por ello, para la primera versión **no recomiendo convertir HAIP en un software fiscal DIAN completo** a menos que exista una razón empresarial fuerte. Es más económico y seguro:

`PMS → Adaptador contable/fiscal → proveedor existente → DIAN`.

Además, DIAN indicó en 2025 que los sistemas de facturación deben adecuarse al servicio que permite generar la factura con tipo/número de identificación y que no debe solicitarse información adicional como teléfono, dirección o RUT para ese propósito; cuando el cliente no está en el servicio, DIAN delimita los datos que pueden solicitarse. Esto debe reflejarse en la UX de checkout para no recolectar información excesiva. [21]

Integración de cerraduras, cámaras, migración y contabilidad

Cerraduras y control de puertas

No compraría ningún hardware nuevo ni escribiría una sola línea de integración definitiva antes de realizar un **levantamiento técnico del sistema existente**.

Hay que obtener:

Dato	Ejemplo
Fabricante	Vingcard, dormakaba, SALTO, otro
Modelo exacto	Controlador/cerradura
Software actual	Nombre + versión

Dato	Ejemplo
Arquitectura	Offline / online / cloud
Servidor/controlador	IP/modelo
Tipo de credencial	tarjeta, PIN, BLE, etc.
SDK/API	Sí/no
Licencia API	Incluida/pagada
Protocolo	REST/SOAP/SDK/otro
Eventos	entrada, apertura, fallo
Apertura remota	soportada/no
PMS existente	cuál
Contacto integrador	proveedor actual

HAIP ya tiene **credential hooks para cerraduras**, así que el diseño adecuado es crear implementaciones de una interfaz `DoorLockProvider`, en lugar de meter llamadas al fabricante directamente dentro de reservas. [22]

Flujo ideal:

```
Check-in / Inicio estancia
        ↓
HAIP genera solicitud de acceso
        ↓
DoorLockProvider
        ↓
Edge Gateway del sitio
        ↓
Sistema del fabricante
        ↓
Credencial / apertura
```

La acción **“abrir puerta ahora”** debe estar separada de “emitir credencial”. Una apertura administrativa debería exigir permiso específico, habitación, usuario, motivo y registro de hora; para determinados roles, incluso PIN de reautenticación.

Entre fabricantes con opciones documentadas para integraciones hoteleras se encuentran:

Fabricante	Ecosistema	Integración oficial relevante	Aproximación
dormakaba	Ambiance	Integración PMS/web services y operación hotelera	Adapter
Vingcard	Visionline / VConnect / Vostio	Integraciones PMS cloud/on-prem	Adapter
SALTO	Space / KS	APIs e integraciones PMS	Adapter
Controlador desconocido actual	Por identificar	Depende de SDK/API	Discovery obligatorio

SALTO documenta APIs para integración de sistemas hoteleros y PMS, incluida su plataforma Space; igualmente existen integraciones PMS en sus ecosistemas cloud. [23]

La existencia de estas APIs **no significa que el equipo instalado actualmente sea compatible**. Modelo, firmware y licencia importan. Por eso la tarea de descubrimiento de hardware es P0.

Si el sistema actual solo permite relés/contactos y no API, aún puede ser posible integrar mediante un gateway industrial, pero eso sería una segunda opción y debe validarse con el fabricante para no comprometer seguridad, operación ni garantía.

Cámaras

Para video recomiendo integrar el **NVR/VMS existente**, no hacer que HAIP se convierta en grabador de video.

Arquitectura:

```
React/Tauri → API autorización → Video Gateway local → NVR/VMS → cámara
```

El PMS decide **si el usuario puede ver la cámara**; el gateway/VMS se encarga del video.

ONVIF Profile T ofrece streaming IP H.264/H.265, eventos y otras capacidades interoperables de video. ONVIF está retirando Profile S para nuevos desarrollos y recomienda mecanismos más seguros asociados a Profile T/TLS; el fin de soporte para Profile S está fijado para el 31 de marzo de 2027. [24]

Para control de acceso físico, ONVIF también define perfiles A, C, D y M; Profile A cubre configuración como credenciales, horarios y reglas de acceso. [25]

Ejemplos de integración de cámaras:

Ecosistema	Interfaces	Recomendación
Cámara/NVR ONVIF Profile T	ONVIF	Primera opción genérica

Ecosistema	Interfaces	Recomendación
Axis	ONVIF + VAPIX	Muy integrable
NVR/VMS propietario	API fabricante	Preferir API del VMS
Cámaras solo Profile S antiguas	ONVIF S	Mantener legado, no diseñar nuevo sistema sobre S

Axis, por ejemplo, publica VAPIX como API abierta para integrar y controlar sus dispositivos y documenta APIs de streaming/eventos. [26]

En la interfaz propondría permisos por **zona**, no simplemente `cameras.read`:

```
camera.reception.view
camera.entrance.view
camera.parking.view
camera.hallways.view
camera.security_admin
```

Y cada acceso registra:

```
user_id
camera_id
zone
started_at
ended_at
device_id
reason (cuando sea exigido)
```

Housekeeping no ve cámaras. Un recepcionista podría ver entrada/parqueadero, pero no necesariamente todos los pasillos. Propietarios/seguridad tienen otra matriz.

Migración de datos

Haría la migración en cuatro pasos.

Inventario de fuentes: sistema actual, Excel, bases de datos, software de cerraduras, listado de habitaciones, tarifas, clientes, productos, cuentas por cobrar, saldos de caja, proveedores y contabilidad.

Normalización: limpiar duplicados de huéspedes, productos y habitaciones; mapear códigos; decidir qué historial realmente merece migrarse.

Carga de prueba: aprovechar el importador CSV de HAIP, que ya soporta mapeo, dry-run y errores por fila, y escribir importadores adicionales para entidades propias. [27]

Cutover: cierre del sistema anterior, exportación final, conciliación, importación, apertura del nuevo PMS y conservación del sistema anterior como archivo de solo lectura durante el periodo acordado.

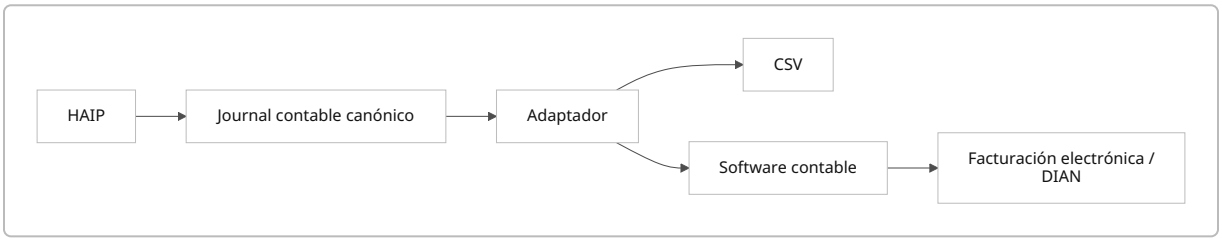
No migraría años de basura "por si acaso". Separaría:

- datos operativos vivos;
- histórico que necesita consulta;
- histórico contable/legal;
- datos innecesarios que no deben replicarse.

Integración contable

HAIP ya exporta diario de ingresos y balance en CSV, por lo que la primera integración puede ser bastante rápida. [28]

El modelo que recomiendo es:



Antes de programar un conector hay que identificar qué usa actualmente el negocio. Si fuera Siigo o Alegra, ambos ofrecen APIs oficiales de integración con sus plataformas; la decisión debe hacerse después de conocer la herramienta real de la empresa. [29]

No acoplaría la lógica PMS directamente a "Siigo". Implementaría:

```
interface AccountingProvider {
  exportSale(...): Promise<Result>;
  exportPayment(...): Promise<Result>;
  exportCreditNote(...): Promise<Result>;
  exportInventoryMovement(...): Promise<Result>;
}
```

y proveedores:

```
CsvAccountingProvider
SiigoAccountingProvider
```

```
AlegraAccountingProvider
OtherAccountingProvider
```

Además, cada exportación necesita `external_id`, estado, reintentos e idempotencia para evitar duplicar comprobantes.

Aplicaciones, módulos, estimaciones y riesgos

Aplicación de recepción

Compararía así las opciones:

Opción	Ventaja	Debilidad	Para Justin/ Jurassic
PWA navegador/ kiosk	Simple de desplegar	Integración local limitada	Buena
Electron	Ecosistema Node muy amplio	Runtime más pesado	Buena
Tauri + React	Web UI + shell nativo liviano	Algo de Rust/bridge	**Preferida**

Tauri permite reutilizar un frontend web y usar el WebView nativo del sistema operativo, lo que encaja bien con React y permite añadir capacidades locales sin meterlas en el navegador. [30]

Electron soporta modo kiosk y sigue siendo una alternativa válida, especialmente si un SDK antiguo de puerta/impresora depende del ecosistema Node/Windows. [31]

Mi arquitectura sería:

```
apps/
  dashboard/            ← HAIP administración
  frontdesk-touch/      ← React dedicado
  staff-pwa/            ← React PWA
  edge-gateway/         ← Servicio local
  desktop-shell/        ← Tauri
```

No convertiría el dashboard existente en el front desk táctil a punta de condicionales. Compartiría componentes y SDK, pero mantendría experiencias separadas.

La recepción debería soportar:

- arranque automático;
- kiosk/full screen;
- recuperación tras reinicio;

• touch targets grandes;
• teclado numérico en pantalla;
• usuario/turno visible siempre;
• sincronización visible;
• indicador **ONLINE/OFFLINE**;
• cero ventanas del navegador;
• actualización controlada;
• cache local cifrada;
• logs locales;
• conexión únicamente al Edge Gateway/API autorizados.

PWA móvil

La PWA debe ser **offline-first para tareas**, utilizando service worker para recursos/cache y almacenamiento persistente local para la cola de cambios. Los service workers permiten interceptar solicitudes y trabajar con estrategias offline; IndexedDB es apropiado para almacenar datos estructurados en el cliente. [32]

No almacenaría en la PWA todo el PMS. Solo:

```
roomAssignments
taskCache
productCatalogSubset
pendingConsumptions
pendingStockMoves
incidents
outbox
```

Debe tener un indicador inequívoco:

`3 cambios pendientes de sincronización`

y permitir reintentar sin que el empleado cree cargos duplicados.

Módulos y esfuerzo

Las siguientes son **estimaciones de ingeniería**, no cotizaciones. Suponen un equipo competente en TypeScript/React/NestJS, conocimiento razonable de HAIP después de la fase de discovery y revisión de código. Un día de desarrollo se toma aproximadamente como 8 horas técnicas.

Módulo	Prioridad	Esfuerzo	Complejidad	Riesgo principal
Fork, tag, build, CI y baseline HAIP	P0	24–40 h / 3–5 d	Media	Cambios upstream
Modelo sitio/unidad multihotel	P0	40–72 h / 5–9 d	Media	Alcance legal/ contable

Módulo	Prioridad	Esfuerzo	Complejidad	Riesgo principal
Roles y permisos propios	P0	40–64 h / 5–8 d	Media	Sobreexposición
Front Desk Touch	P0	80–140 h / 10–18 d	Alta	UX bajo presión
Motor estancias cortas	P0	120–200 h / 15–25 d	Alta	Reglas tarifarias
Adaptación housekeeping	P0	32–56 h / 4–7 d	Media	Estados inconsistentes
Inventario por habitación/ minibar	P0	120–180 h / 15–23 d	Alta	Conciliación stock/ folio
PWA operacional	P0	72–120 h / 9–15 d	Alta	Offline/reintentos
Checkout/descuentos/ comisiones	P0	80–140 h / 10–18 d	Alta	Fraude/reglas negocio
Caja, turnos y auditoría ampliada	P0	64–100 h / 8–13 d	Alta	Conciliación
Descubrimiento + adapter cerraduras	P0	80–200 h / 10–25 d	Alta	API desconocida
Portal central propietario	P1	64–120 h / 8–15 d	Media	Métricas/alcance
Fidelización/promociones	P1	48–80 h / 6–10 d	Media	Privacidad/reglas
Video gateway + permisos	P1	80–140 h / 10–18 d	Alta	Protocolo/seguridad
Edge/offline/sincronización	P1	160–260 h / 20–33 d	Muy alta	Conflictos/red
Exportador/adapter contable	P1	32–100 h / 4–13 d	Media/alta	Software destino
Integraciones cumplimiento Colombia	P1	80–160 h / 10–20 d	Alta	Reglas externas
Migración	P1	40–120 h / 5–15 d	Variable	Calidad de datos
DevOps/backups/ observabilidad/security	P0	80–140 h / 10–18 d	Alta	Operación 24/7
QA/E2E/failover/UAT	P0	120–200 h / 15–25 d	Alta	Escenarios físicos

El orden importa más que el número de horas. **No desarrollaría cámaras primero.** El dinero y control operacional están en: habitación → estancia → puerta → consumo → cobro → caja → aseo → inventario → auditoría.

El rango agregado del alcance completo está alrededor de **1.300–2.400 horas**, dependiendo especialmente del hardware y de cuán sofisticado deba ser el modo offline.

Con un solo desarrollador, intentar hacer todo simultáneamente sería un riesgo. Con dos desarrolladores full-stack fuertes más QA/DevOps parcial puede plantearse un proyecto por etapas de varios meses.

Plan de desarrollo, pruebas, despliegue y seguridad

Propongo un proyecto progresivo, sin intentar reemplazar absolutamente todo el primer día.

Etapa	Semanas orientativas	Resultado
Discovery técnico/operativo	1–2	Hardware, flujos, datos, HAIP congelado
Fundación	2–4	HAIP seguro, propiedades, usuarios, roles
MVP operacional	5–10	Touch + hotel + motel + folio + caja
Inventario/PWA	8–13	Minibar, aseo, stock, móvil
Hardware	10–15	Cerraduras + video PoC/producción
Resiliencia	12–17	Edge/offline/reconciliación
Contabilidad/compliance	14–18	Exportación e integraciones
Piloto Justin o Jurassic	18–20	Operación real paralela
Segundo sitio	20–22	Rollout
Hardening	22–24	Seguridad, métricas, mejoras

Las etapas pueden solaparse para un equipo de varias personas. Yo haría **piloto en una sola ubicación**, con el sistema antiguo disponible en paralelo, antes de desplegar en ambas.

Estrategia de ramas

```
upstream/TelivityAI/haip
        ↓
vendor/haip-v3.0.7
        ↓
main
 ├─ feat/short-stay
 ├─ feat/frontdesk-touch
 ├─ feat/room-inventory
```

```
├─ feat/driver-referrals
├─ feat/edge-gateway
└─ integrations/...
```

Cada actualización de HAIP se probaría en staging y se fusionaría deliberadamente. Debido al ritmo reciente de releases del proyecto, no configuraría actualizaciones automáticas del core en producción. La release más reciente observada es v3.0.7 del 19 de octubre de 2026. [33]

Matriz de pruebas crítica

Tarifas motel: límites exactos 15/30/60, segundos antes/después del corte, gracia, extensiones, medianoche, promociones, cambio de habitación y tarifa manual.

Finanzas: doble clic en cobrar, caída de Internet después del cobro, reintento, devolución, anulación, descuento, múltiples métodos, diferencia de caja.

Inventario: consumo duplicado, reposición, producto agotado, conteo físico, devolución, ajuste, PWA offline.

Puertas: check-in, extensión, cambio de habitación, checkout, revocación, apertura administrativa, caída del servidor, caída de Internet, credencial expirada.

Permisos: un recepcionista no debe ver cámaras no autorizadas; housekeeping no puede leer finanzas; un gerente Justin no puede acceder a Jurassic; ningún usuario puede elevar sus permisos manipulando peticiones.

Offline: cortar WAN durante check-in, durante consumo, durante checkout y justo antes de sincronización; reactivar conexión y confirmar que no aparecen duplicados.

Concurrencia: dos recepcionistas intentando ocupar la misma habitación.

Migración: totales antes/después, habitaciones, saldos, clientes, inventario.

E2E principal:

```
Cliente llega
→ habitación
→ estancia
→ puerta
→ consumos
→ cierre
→ pago
→ factura
→ comisión
→ caja
```

```
→ habitación sucia
→ aseo
→ reposición
→ habitación disponible
```

Ese flujo debe probarse completo repetidamente con usuarios reales de recepción.

Checklist de seguridad previo a producción

- [ ] `AUTH_ENABLED=true`.
- [ ] Keycloak en configuración productiva.
- [ ] MFA para propietaria y administradores.
- [ ] Usuarios individuales, sin cuentas compartidas.
- [ ] Permisos por unidad/sitio.
- [ ] Cámara por zona y rol.
- [ ] Apertura manual de puerta auditada.
- [ ] Reautenticación para operaciones críticas.
- [ ] Secretos fuera del frontend/repo.
- [ ] Contraseñas de cámaras solo en gateway/VMS.
- [ ] Red de huéspedes segregada.
- [ ] Cámaras y puertas no expuestas directamente a Internet.
- [ ] Cifrado de comunicaciones.
- [ ] Disco/cache local protegido.
- [ ] Backup automático de PostgreSQL.
- [ ] Prueba real de restauración de backup.
- [ ] Retención diferenciada por tipo de dato.
- [ ] Logs de cambios administrativos.
- [ ] Log de cámara y puerta.
- [ ] Protección frente a reintentos/doble cobro.
- [ ] Idempotency keys en integraciones.
- [ ] Dependencias bloqueadas mediante lockfile.
- [ ] Actualizaciones probadas en staging.
- [ ] Escaneo de dependencias/secretos en CI.
- [ ] Política de tratamiento de datos.
- [ ] Avisos de videovigilancia.
- [ ] Consentimientos comerciales separados.
- [ ] Plan de respuesta a incidentes.
- [ ] Procedimiento cuando Internet falla.
- [ ] Procedimiento cuando HAIP central falla.
- [ ] Procedimiento cuando el sistema de puertas falla.

El README de HAIP describe aislamiento por `property_id`, validación de JWT, CORS, cabeceras de seguridad, protección SSRF, rate limiting, auditoría y otros controles, pero el propio proyecto advierte que una instalación debe someterse a su propia revisión de seguridad. Esa advertencia es especialmente importante al conectar un PMS con infraestructura física como cerraduras y cámaras. [18]

Entregables preparados para trabajar con Claude

Para desarrollar esto con Claude Code no le daría simplemente un prompt como “modifique HAIP para mi hotel”. El proyecto debe convertirse primero en documentación contractual para que el agente no invente reglas de negocio.

De hecho, el propio repositorio de HAIP establece entre sus pautas de contribución que **no se debe inventar lógica del dominio hotelero** y que los cambios de lógica deben ir acompañados por pruebas. [9]

El paquete que prepararía tendría esta estructura:

```
/docs/project/
├── 00-vision.md
├── 01-business-context.md
├── 02-current-haip-baseline.md
├── 03-domain-model.md
├── 04-multi-property-model.md
├── 05-short-stay-spec.md
├── 06-frontdesk-touch-ux.md
├── 07-mobile-pwa-spec.md
├── 08-room-inventory-spec.md
├── 09-driver-referrals-spec.md
├── 10-access-control-integration.md
├── 11-video-integration.md
├── 12-accounting-integration.md
├── 13-offline-sync.md
├── 14-colombia-compliance.md
├── 15-rbac-matrix.md
├── 16-audit-events.md
├── 17-api-contracts.md
├── 18-migration-plan.md
├── 19-test-plan.md
├── 20-deployment.md
└── ADR/
    ├── ADR-001-central-db.md
    ├── ADR-002-edge-gateway.md
    ├── ADR-003-tauri.md
    └── ADR-004-business-units.md
```

Historias de usuario de máxima prioridad:

ID	Historia	Criterio esencial
FD-001	Como recepcionista quiero ver todas las habitaciones	Estado actualizado en tiempo real
FD-002	Quiero iniciar estancia corta	Tarifa calculada en servidor
FD-003	Quiero extender una estancia	Recalcula sin duplicar cargos
FD-004	Quiero abrir habitación	Solo permiso autorizado + audit
FD-005	Quiero cerrar cuenta	No permite cierre con consumos pendientes
HK-001	Como aseo quiero ver habitaciones asignadas	No veo datos financieros
HK-002	Quiero cargar minibar	Actualiza stock + folio una vez
HK-003	Quiero marcar habitación limpia	Propaga estado en tiempo real
INV-001	Quiero saber stock por habitación	Cuadra con movimientos
CASH-001	Quiero abrir/cerrar turno	Diferencia registrada
MGR-001	Quiero autorizar descuento	Usuario, motivo y monto auditados
REF-001	Quiero registrar conductor	Comisión enlazada a estancia
OWN-001	Quiero consolidar hoteles	Vista de todas las unidades
SEC-001	Quiero ver cámaras autorizadas	Solo zonas concedidas
AUD-001	Quiero consultar acciones sensibles	Registro no editable por operador
OFF-001	Quiero seguir operando sin WAN	Cola local visible
OFF-002	Quiero reconectar	Sin duplicación de operaciones

Ejemplo de criterio de aceptación que Claude debería recibir:

```
Feature: Consumo de minibar desde PWA offline

Scenario: Registrar un consumo sin conexión
  Given la habitación 205 está ocupada
  And el empleado tiene permiso de inventario de habitación
  And la PWA está sin conexión a Internet
  When registra 2 unidades de cerveza
  Then la PWA debe guardar el movimiento en la outbox local
  And debe mostrarlo como "pendiente de sincronización"
  And no debe permitir generar dos operaciones con el mismo idempotencyKey

Scenario: Recuperación de conexión
  Given existe el consumo pendiente anterior
```

```
  When vuelve la conectividad
  Then el evento debe sincronizarse una sola vez
  And el inventario de la habitación debe disminuir en 2
  And el folio debe recibir un único cargo
  And la auditoría debe registrar al empleado original
```

Y otro para el control más sensible:

```
Feature: Apertura administrativa de una puerta

Scenario: Recepcionista sin permiso
  Given el usuario tiene rol RECEPCIONISTA
  And no tiene permiso door.emergency_open
  When intenta abrir una habitación desde el PMS
  Then la API debe responder 403
  And no debe enviarse ningún comando al controlador

Scenario: Supervisor autorizado
  Given el usuario tiene permiso door.emergency_open
  When solicita abrir la habitación 205
  And confirma su PIN
  And introduce un motivo
  Then se debe enviar el comando al Edge Gateway
  And registrar usuario, habitación, hora, dispositivo y motivo
```

Definition of Done por módulo:

1. migración de base incluida;
2. API documentada;
3. permisos aplicados en backend, no únicamente UI;
4. audit event definido;
5. errores y reintentos contemplados;
6. pruebas unitarias;
7. pruebas de integración;
8. escenario E2E;
9. modo offline definido cuando corresponda;
10. logging/observabilidad;
11. documentación;
12. rollback posible.

El roadmap que entregaría a Claude sería deliberadamente secuencial:



La prioridad final que propondría para el negocio es muy clara:

Primero: saber exactamente qué habitación está ocupada, por quién fue abierta, qué tarifa tiene, quién autorizó descuentos, cuánto consumió, cuánto se cobró, quién recibió el dinero y cómo quedó la habitación.

Después: inventario, aseo, comisiones, cierres y control remoto.

Luego: offline robusto, cámaras integradas, automatización contable y fidelización avanzada.

Esa secuencia convierte HAIP no solo en un PMS para Justin y Jurassic, sino en una **plataforma central de control operacional y financiero para la familia**, manteniendo la posibilidad de añadir posteriormente otros hoteles, sucursales o unidades de hospedaje sin rehacer la arquitectura.

---

1 3 5 6 8 9 10 11 12 14 15 16 17 18 22 27 28 GitHub - TelivityAI/haip: HAIP (Hotel AI Platform) — Open-source, API-first hotel PMS. TypeScript/Node.js/NestJS. · GitHub
https://github.com/TelivityAI/haip

2 haip/LICENSE at main · TelivityAI/haip · GitHub
https://github.com/TelivityAI/haip/blob/main/LICENSE

4 Preguntas frecuentes relacionadas con formalización turística | MINCIT - Ministerio de Comercio, Industria y Turismo
https://www.mincit.gov.co/minturismo/analisis-sectorial-y-promocion/preguntas-frecuentes-relacionadas-con-formalizacio

7 haip/apps/api/package.json at main · TelivityAI/haip · GitHub
https://github.com/TelivityAI/haip/blob/main/apps/api/package.json

13 haip/docs/deployment.md at main · TelivityAI/haip · GitHub
https://github.com/TelivityAI/haip/blob/main/docs/deployment.md

19 Política de Tratamiento de Datos Personales
https://sedeelectronica.sic.gov.co/politica-de-tratamiento-de-datos-personales?utm_source=chatgpt.com

20 ¿Cómo elegir la solución de facturación?
https://www.dian.gov.co/Prensa/Paginas/BlogDetails.aspx?DianId=5&utm_source=chatgpt.com

21 Prensa Comunicado de Prensa No. 026
https://www.dian.gov.co/Prensa/Paginas/NG-Comunicado-de-Prensa-026-2025.aspx

23 Smart Solutions for Hotels: Explore the Salto Integrations
https://saltosystems.com/en/blog/integrations_endless_possibilities/?utm_source=chatgpt.com

24 Profile T
https://www.onvif.org/profiles/profile-t/?utm_source=chatgpt.com

25 ONVIF Profiles
https://www.onvif.org/profiles/?utm_source=chatgpt.com

26 VAPIX® | Axis developer documentation
https://developer.axis.com/vapix/?utm_source=chatgpt.com

29 Siigo API
https://developers.siigo.com/?utm_source=chatgpt.com

30 What is Tauri?
https://v2.tauri.app/start/?utm_source=chatgpt.com

31 BaseWindow
https://electronjs.org/docs/latest/api/base-window?utm_source=chatgpt.com

32 Service workers
https://web.dev/learn/pwa/service-workers?utm_source=chatgpt.com

33 Releases · TelivityAI/haip · GitHub
https://github.com/TelivityAI/haip/releases